ASSIGNMENT-4

ASSIGNMENT 4: Files, Exceptions, and Errors in Python 💻

Aim and Objectives

Aim: To develop Python programs that demonstrate proficiency in file handling operations (reading, writing, appending) and robust error handling, specifically for FileNotFoundError.

Objectives:

1. To successfully open and read the content of a text file line by line.
2. To implement a try-except block to gracefully handle a FileNotFoundError.
3. To take user input and write it to a new file.
4. To append additional data to an existing file.
5. To read and display the complete final content of a file after writing and appending.

Requirements

- A Python interpreter (version 3.x is recommended).
- A code editor or IDE (e.g., VS Code, PyCharm).
- A sample.txt file (to test Task 1's successful file read) with a few lines of text.

Theory

File handling in Python involves using built-in functions like open() to interact with files. The open() function takes the file name and a mode as arguments:

- 'r' (Read): Opens a file for reading. This is the default mode. If the file doesn't exist, a FileNotFoundError is raised.
- 'w' (Write): Opens a file for writing. Creates the file if it doesn't exist, or truncates (clears) the file if it does exist.
- 'a' (Append): Opens a file for appending. Creates the file if it doesn't exist. Data written is added to the end of the file.

Exception Handling is crucial for writing robust programs. Python uses the try...except block to anticipate and manage potential runtime errors. For instance, try to open and read a file, and if a FileNotFoundError occurs, the code within the corresponding except block is executed, allowing the program to continue running gracefully. The with open(...) construct is generally preferred as it ensures the file is automatically closed, even if errors occur.

Task 1: Read a File and Handle Errors

Problem Statement

Write a Python program that:

1. Opens and reads a text file named sample.txt.
2. Prints its content line by line.
3. Handles errors gracefully if the file does not exist.

Algorithm / Steps

1. Start a try block to anticipate a file-related error.
2. Within the try block, use with open('sample.txt', 'r') as file: to open the file for reading.
3. Print a message indicating the content is being read.
4. Use a for loop to iterate through the file object, reading and printing each line.
5. Follow the try block with an except block specifically catching FileNotFoundError.
6. If the error occurs, print the custom error message as specified in the expected output.

Code (task1_read_file.py)

Python

```
def read_and_handle_file(filename="sample.txt"):
    """Opens, reads, and prints file content line by line, handling FileNotFoundError."""
    try:
        # Use 'with' statement for automatic file closing
        with open(filename, 'r') as file:
            print("Reading file content:")
```

```
            # Read and print content line by line
            line_number = 1
            for line in file:
                # Use .strip() to remove potential trailing newlines
                print(f"Line {line_number}: {line.strip()}")
                line_number += 1

    except FileNotFoundError:
        # Gracefully handle the error if the file is not found
        print(f"Error: The file '{filename}' was not found.") # [cite: 7, 9]

# A necessary 'sample.txt' file with two lines should exist
# to test the successful path.
if __name__ == "__main__":
    # Test case 1: File exists (assuming you've created a sample.txt)
    print("--- Test 1: File Exists ---")
    # For a successful run, sample.txt should contain:
    # This is a sample text file.
    # It contains multiple lines.
    read_and_handle_file("sample.txt")

    print("\n--- Test 2: File Does Not Exist ---")
    # Test case 2: File does not exist
    read_and_handle_file("nonexistent_file.txt")
```

Output

Case 1: sample.txt exists (Content: "This is a sample text file.\nIt contains multiple lines.")

```
--- Test 1: File Exists ---
Reading file content:
Line 1: This is a sample text file.
Line 2: It contains multiple lines.
```

Case 2: sample.txt does NOT exist

```
--- Test 2: File Does Not Exist ---
Error: The file 'nonexistent_file.txt' was not found.
```

Task 2: Write and Append Data to a File

Problem Statement

Write a Python program that:

1. Takes user input and writes it to a file named output.txt.
2. Appends additional data to the same file.
3. Reads and displays the final content of the file.

Algorithm / Steps

1. Prompt the user for the **initial text** to be written and store it in a variable.
2. Use with open('output.txt', 'w') to open the file in **write mode** ('w') and write the initial text. Print a success message.
3. Prompt the user for the **additional text** to be appended and store it in a variable.
4. Use with open('output.txt', 'a') to open the file in **append mode** ('a') and write the additional text (preceded by a newline character \n). Print a success message.
5. Use with open('output.txt', 'r') to open the file in **read mode** ('r') to display the final content.
6. Print a header, read the entire content using .read(), and print it.

Code (task2_write_append_read.py)

Python

```
def write_append_and_read(filename="output.txt"):
    """
    Takes user input, writes it to a file, appends more data,
    and then displays the final content.
    """

    # 1. Take initial user input and write to file ('w' mode)
    initial_text = input("Enter text to write to the file: ") #

    try:
        with open(filename, 'w') as file:
            file.write(initial_text + "\n") # Write with a newline
        print(f"Data successfully written to {filename}.")

        # 2. Take additional user input and append to file ('a' mode)
        additional_text = input("Enter additional text to append: ") #

        with open(filename, 'a') as file:
            file.write(additional_text + "\n") # Append with a newline
        print("Data successfully appended.")

        # 3. Read and display the final content of the file ('r' mode)
        print("\nFinal content of output.txt:") #
        with open(filename, 'r') as file:
```

```
            final_content = file.read()
            print(final_content.strip()) # strip() to remove final trailing newline

    except Exception as e:
        # Catch other potential errors (e.g., PermissionError)
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    write_append_and_read()
```

Output

(Based on the example input from the problem statement)

User Input:

- Enter text to write to the file: Hello, Python!
- Enter additional text to append: Learning file handling in Python.

Console Output:

```
Enter text to write to the file: Hello, Python!
Data successfully written to output.txt.
Enter additional text to append: Learning file handling in Python.
Data successfully appended.

Final content of output.txt:
Hello, Python!
Learning file handling in Python.
```

Learning Outcomes

Upon completion of this assignment, the following key concepts and skills are mastered:

- **File Modes:** Distinguishing between and correctly using the 'r' (read), 'w' (write), and 'a' (append) file modes.

- **Context Management:** Utilizing the **with open(...)** statement for reliable and automatic closing of file resources, which is a Python best practice.
- **Iteration:** Reading a file line by line using a for loop, which is memory efficient for large files.
- **Error Handling (Exceptions):** Implementing the **try...except** block to anticipate and gracefully handle specific exceptions, notably the FileNotFoundError, preventing program crashes and providing user-friendly feedback.
- **Program Flow:** Structuring a complete program flow that integrates user input, file writing, modification (appending), and final verification (reading).